

Report of Team McMurdo: Next-Gen Programming - AI-Powered Software Programming with Deep Learning

1st Priyam Turakhia
BSc Information Systems
TU München
ge58wap@tum.de

2nd Julian Pins
BSc Informatics
TU München
ge87yos@mytum.de

3rd Alena Bobovich
BSc Information Systems
TU München
ge48haq@mytum.de

4th Frytz Martinek
BSc Information Systems
TU München
ge84kax@mytum.de

Abstract—This project explores the use of fine-tuned large language models (LLMs) to detect and modernize deprecated NumPy code. Using parameter-efficient fine-tuning (LoRA) and a retrieval-augmented generation (RAG) setup with ChromaDB, the system combines model training with context retrieval from recent documentation. The result is a lightweight VS Code extension for automated code updates suggestions, serving as a starting point for future support of further libraries.

I. INTRODUCTION

Keeping ageing code aligned with evolving APIs is an everyday headache. The crux is not just updating libraries, but reacting to their deprecations. An empirical study of 297,254 Java projects found that the most common response to an API deprecation was *no response at all*: 88% of usages stayed unchanged [1]. When a later release finally removes or alters the deprecated call, builds break and teams scramble for migration guidance.

Our project targets that precise pain in a narrow, common setting: NumPy upgrades. We built a lightweight VS Code plug-in that flags deprecated NumPy functions in the user's code and surfaces the officially recommended replacement, plus a short rationale.

Goals:

- G1 Model effectiveness:** Show that a task-specific fine-tuning, combined with RAG, can empower small models in detecting and modernizing deprecated NumPy functionality.
- G2 Practical utility:** Ship a VS Code extension that catches deprecated NumPy calls in everyday scripts and suggests the official replacement so the developer can migrate with a single copy-and-paste.

The report first details our dataset construction, retrieval integration, training pipeline, and system architecture. We then discuss model hyperparameters and quantization choices, present before/after evaluation results, and close with limitations and lessons learned.

II. BACKGROUND AND RELATED WORK

A. Introduction

With the rapid emergence of highly capable general-purpose large language models (LLMs) such as GPT-4 and LLaMA in the recent years, the expectations for LLM performance have risen significantly. These models demonstrate great capabilities across a wide range of tasks and have set the benchmark for what users and developers expect from LLMs. However, creating such frontier models requires vast computational resources, massive training datasets and months of engineering effort, resources which most individuals or smaller research teams do not possess.

B. Fine-tuning

As a result, there has been a major shift in LLM development, which is largely associated with the technique called fine-tuning. Building new models was once a task reserved for large organizations with massive computational resources. With this new shift, even individuals or small teams can adapt existing open-source models to specific tasks using local machines with limited hardware.

Base models like GPT-4 are pretrained on enormous corpora, such as web data, books, and code. During pretraining, the model learns general patterns in language, such as grammar and reasoning, by predicting the next token in a sentence. In fine-tuning, the training of a pretrained large language model is continued on a smaller, task-specific dataset. This training process refines the model's parameters to better suit the new specific task, such as answering questions about a particular domain, in our case NumPy code modernization. Through fine-tuning, much smaller models can outperform even the best models on specific tasks [2].

C. Parameter-efficient Fine-Tuning and LoRA

Although base models used for fine-tuning are normally much smaller than frontier ones, updating all parameters during fine-tuning requires considerable computational resources and increases the risk of the so-called "catastrophic forgetting"

[3]. To address this, recent research has shifted toward optimizing the fine-tuning approach itself - this is how Parameter-Efficient Fine-Tuning (PEFT) [2] was proposed. In PEFT, only a portion of parameters is updated, and the rest of the parameters is shared across different tasks. Research shows that PEFT is efficient and surprisingly effective in fine-tuning models. Apart from reducing the computational and memory cost associated with full fine-tuning, PEFT can improve the stability and performance of many prevalent works [2].

A promising approach for PEFT is the Low-Rank Adaptation of Large Language Models (LoRA) [4], that has become popular due to its effectiveness. LoRA works by injecting low-rank trainable matrices into the model’s weight paths (e.g. in attention layers), such that during training, only these additional matrices are updated, while the original weights remain frozen [4]. This approach reduces the number of trainable parameters drastically, while the performance of the resultant model is still comparable to the performance of fully fine-tuned models [4]. This allows for fine-tuning relatively larger models using consumer hardware.

D. Retrieval-Augmented Generation

Another trend that caught our attention is Retrieval-Augmented Generation (RAG) [5]. While pretrained models are limited to the knowledge stored in their parameters, RAG systems aim to extend the model’s knowledge with external, up-to-date, domain-specific context. RAG systems are designed to retrieve relevant information from external knowledge bases or document stores before sending the prompt to the LLM. In this setup, a query formed from user input is first converted into an embedding vector (numerical vector representation of text that captures its semantic meaning) [6]. A vector database, like Chroma that we will use for our project, contains precomputed embeddings of our documents. The query vector is then compared to the document vectors using a similarity metric, often cosine similarity, like in our case. The database returns the most similar vectors closest to the query. The retrieved documents are then added as context to the original prompt sent to the LLM. This gives the model additional domain-specific knowledge that it wasn’t trained on [5], allowing it to stay up-to-date with newly added information, such as NumPy deprecations in our case, without being retrained.

Therefore, understanding RAG systems, fine-tuning and the LoRA approach in particular was a starting point for our project. Both techniques, RAG and fine-tuning, can be used in combination to improve model performance for our specific task. These techniques have been shown to complement each other [5], so that their combination can be used in our project to achieve best results. In our project, we apply fine-tuning to teach the model about modernizing functionality and having structured output, and use RAG to give it relevant context about deprecations beyond its training data.

III. DEVELOPMENT PROCESS

We adopted an agile workflow inspired by Scrum. Priyam Turakhia served as Scrum Master, compiled the initial product backlog, and facilitated task allocation across the team.

A. Phase 1—Backend & Data-Scraping

Objective: Collect the relevant data and build a backend prototype, including an initial RAG pipeline.

TABLE I
PHASE 1 RESPONSIBILITIES

Responsibility	Team Members
Backend, RAG expansion Data scraping/ generation & evaluation metrics	Priyam Turakhia; Alena Bobovich Julian Pins; Frytz Martinek

B. Phase 2—Front-end, Fine-Tuning & Model Integration

Objective: Fine-tune and evaluate candidate models and develop VS-Code extension.

TABLE II
PHASE 2 RESPONSIBILITIES

Responsibility	Team Members
Model fine-tuning & evaluation	Entire team
Model integration (GGUF conversion, server setup)	Priyam Turakhia
Front-end (VS Code extension)	Julian Pins

C. Sprint Cadence & Ceremonies

Each phase was broken into multiple sprints, each focusing on a single deliverable (e.g. RAG pipeline, evaluation run, or front-end code actions).

At the end of every sprint, we held a short review and retrospective to: demo completed work, surface blockers, discuss solutions and reprioritise the backlog for the next sprint.

This rhythm kept progress transparent and allowed us to adapt quickly to new insights while maintaining a steady delivery pace.

IV. METHOD

A. Scope and Assumptions

This project limits itself to one highly-relevant case of API drift: deprecated NumPy functions and keywords. We track changes announced between numpy 1.4.0 and numpy 2.3.0; anything outside that range (SciPy, Polars, Pandas etc.) is left for future work. The prototype appears as a VS Code extension that lets the developer select a fragment of code in a .py file. The marked snippet is then analysed. If the analysis finds a legacy symbol such as `np.unique1d`, the tool highlights the occurrence and offers the snippet back with the modern alternative (`np.unique`) together with a brief explanation. The

tool never edits code automatically; acceptance or dismissal is entirely up to the developer.

Several practical assumptions keep the problem tractable:

- Python 3.8–3.12 and desktop VS Code on Windows, macOS, or Linux.
- NumPy is imported under common aliases (or none), and the highlighted code is itself parseable without syntax errors, allowing an AST walk.

Our primary goal is to measure how much fine-tuning improves detection and replacement accuracy, building a production-grade refactoring assistant is outside the present scope.

B. Dataset

1) *Unsuccessful Commit Scraping*: Our initial approach was to mine large open-source repositories with heavy NumPy usage. We scraped their commit histories, retaining only those commits in which both the pre- and post-change code referenced NumPy functions. We then filtered the commit messages for keywords such as deprecated function or removed function. For every surviving commit, we organized the pre-change snippet, post-change snippet, message, and related metadata into a structured record. Although this pipeline uncovered some valuable examples, it yielded less than 10% usable data, with most of them centered on just a handful of functions.

2) *Data generation using Gemini*: We next built a deprecated function list by going through the NumPy release notes and reading Stack Overflow posts tagged numpy and deprecated. This yielded 40 deprecated features, including functions, parameters and datatypes. For each symbol, we asked Gemini to generate five to eight runnable code snippets from each symbol. The procedure produced 248 validated examples.

3) *Data Format*: Table III shows all the fields we generated, although for the final training, we only used the fields input, output and context.

- **input** – the original (pre-change) code fragment containing the deprecated call.
- **context** – a short natural-language sentence describing why the change is required and what to do instead.
- **output** – the refactored (post-change) snippet that fulfils the same functionality without the deprecated function.

TABLE III
DATASET SCHEMA (KEY FIELDS).

Field	Example
input	scalar_val = np.asscalar(np.array([42]))
package	numpy
version	1.16.0
context	The function ‘numpy.asscalar’ is deprecated since NumPy 1.16.0. Use ‘numpy.ndarray.item’ instead.
functionality	Convert an array of size 1 to its corresponding Python scalar.
output	scalar_val = np.array([42]).item()

C. Data Pipeline

Because the Gemini prompts already yield examples in the exact “old → new” JSON schema described in Section IV-B, the post-processing we needed is minimal. As for our RAG pipeline, we followed a three-stage script to populate ChromaBD and prepare the material for retrieval at runtime.

Step 1: Chunk extraction

- Extract self-contained chunks from the official NumPy release notes of the past five years and accompanying code snippets for later retrieval.

Step 2: Embedding

- Convert every chunk into a dense vector embedding, preserving its metadata for downstream lookup in the vector database.

Step 3: Vector store

- Persist embeddings and metadata in ChromaDB, which supports cosine-similarity search and serves relevant context at runtime.

D. Model Training

a) First challenges:

Having collected data for supervised learning, we moved on to fine-tuning LLMs. Since our team was limited in local GPU resources, we decided to use Google Colab as an environment for the fine-tuning process. Google Colab is a cloud-based platform that allows users to run Python code with free or paid access to GPUs and TPUs and train models without the need for powerful local hardware. However, even with Google Colab’s resources, both free and Pro versions, fine-tuning a full LLM is too memory intensive. This is why it was crucial to optimize the fine-tuning pipeline by adopting PEFT, specifically LoRA [4]. As a further optimization, we restricted ourselves to LLMs with no more than 3 billion parameters. Despite these optimizations, the free T4 GPUs provided by Google Colab were not enough for us, sometimes leading to RAM crashes. The final solution we came up with was applying 4-bit quantization with the bitsandbytes library [7] during fine-tuning. This method is known as QLoRA [8], which is a fine-tuning approach that reduces memory usage drastically without sacrificing performance [8]. With these optimization techniques, we successfully fine-tuned multiple LLMs and focused on five of them: Gemma-2B, Phi-2, Qwen-0.5B, Qwen-1.8B and TinyLlama-1.1B. Other models were either too large, or turned out to be not instruction tuned.

Another problem we faced during fine-tuning in Google Colab was the constant occurrence of dependency conflicts and incompatibilities between all the different libraries that we used. Since running the script requires all the necessary libraries to be imported successfully, we had to spend a significant amount of time resolving version mismatches before we could even execute the actual fine-tuning process. Searching for a solution online was difficult as these libraries are frequently updated and finding up-to-date guidance was a challenge. After a lot of trial and error, we found suitable

combinations of needed package versions, and, most importantly, restarting the Colab runtime after that also resolved many import errors. With this workaround, we could finally run the rest of the scripts.

During model training, we discovered that the model returned empty outputs. After searching for possible causes, we found that this behavior was likely due to a high learning rate, which caused the model's weights to be updated too aggressively. This might destroy valuable knowledge gained from pretraining and lead the model to a state where the mathematically safest way to minimize loss is to output the end-of-sequence token immediately. Reducing the learning rate solved the problem.

b) Prompt engineering:

The specific goal of our fine-tuning pipeline was to train the model to detect outdated NumPy code in a given code snippet and suggest modern NumPy equivalents in a specific format, while keeping the logic and formatting in the original code. To achieve this, we had to design a suitable prompt for fine-tuning, which turned out to be challenging. Smaller models, due to their limited memory and reasoning ability, often struggle with ambiguity or multi-step instructions unless they are stated explicitly. They may hallucinate replacements or apply incorrect ones if the instructions are not tightly constrained. Our first prompt idea was:

"Replace deprecated functions in the following NumPy code".

Using this prompt resulted in hallucinated deprecations and unusable outputs that contained irrelevant text, tests for the code or unnecessary imports. After trying out different prompts and observing their impact on the model performance, we came up with a direct, successful system prompt for supervised learning:

"You are a Python code refactoring tool for NumPy. Your task is to replace only the deprecated functions in the given code snippet with their modern equivalents. Your response must be structured with two markdown sections:

- 1) A **Refactored Code** section containing ONLY the updated Python code block. Do not change the code's logic. Do not add imports. Do not add comments.
- 2) A **Deprecation Context** section containing a brief explanation of the deprecation.

If no functions are deprecated, return the original code and state that no changes were needed in the context section".

With this final prompt, the fine-tuned model returned more meaningful outputs that matched the required format most of the time.

c) Model exporting:

The final challenge of the fine-tuning pipeline was exporting the resulting models in a format that could be efficiently loaded on our local computers. Even 2B parameter models are memory-intensive, which makes direct loading on standard

hardware difficult. We tried converting the fine-tuned models into the GGUF format, which is optimized for fast and memory-efficient model loading. However, despite the benefits of the GGUF file format, converting fine-tuned models into this format was significantly more difficult than expected, mostly due to documentation gaps.

After completing the training, we merged the fine-tuned LoRA adapter back into the full base model. The result is a de-quantized, specialized language model that can detect and modernize legacy NumPy code. Finally, we pushed the de-quantized model to Hugging Face.

E. System Architecture

In this section, we describe how our system works behind the scenes to detect and update deprecated NumPy code using a combination of a fine-tuned LLM and the RAG approach.

At the heart of our system are the two fine-tuned language models, Gemma-2B and TinyLlama-1.1B, that we selected among the models that we have fine-tuned and evaluated. As described earlier, our fine-tuned models have been trained specifically to handle NumPy deprecations.

To support our models, we use a RAG pipeline that helps the model by providing useful deprecation context from NumPy docs. When the user selects a piece of code in the VS Code extension, it is first sent to our backend. There, we search for relevant information in the vector database ChromaDB [9]. To fill the vector database with relevant information, we populated it with official NumPy release notes from the past five years and NumPy code chunks. To do that, we used an LLM-based scraper to create a .md file of all deprecations from version 1.4 to version 2.3. One of the reasons why ChromaDB is well-suited for our project is that it stores not only the vector embeddings (numerical vector representations of text) [6], but also supports metadata alongside each vector [9]. This means we don't need to use a traditional database for the metadata, ChromaDB handles everything we need in one place.

After extracting the most relevant information from ChromaDB, we build a prompt which includes the original code and the retrieved documentation or code snippets from the knowledge database. The full prompt is then passed to our fine-tuned model, which analyzes the code, rewrites deprecated parts, and explains what changes were made.

Finally, the updated code and the reasoning are returned and rendered to the VS Code extension, allowing the user to view, understand and apply the suggestions. Figure 1 shows a schematic overview of how our extension interacts with the RAG system and the LLM.

F. Resulting Software - The VS Code Extension

To make our fine-tuned model easily accessible to developers and avoid the need for context switching, we built a Visual Studio Code extension with simple functionality. The user highlights a block of Python code in the editor. If the user right-clicks on the highlighted code block and selects "Modernize NumPy code" in the pop-up menu, or just uses our shortcut Ctrl+M, a side panel opens within VS Code.

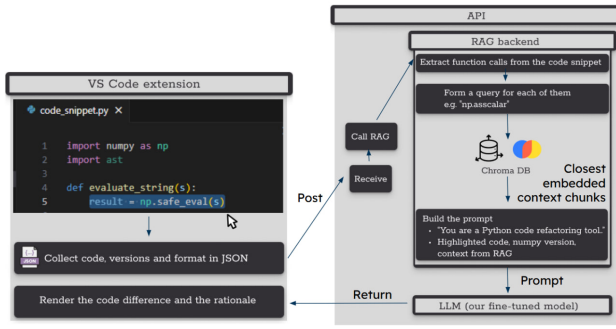


Fig. 1. Interaction of API, RAG and LLM.

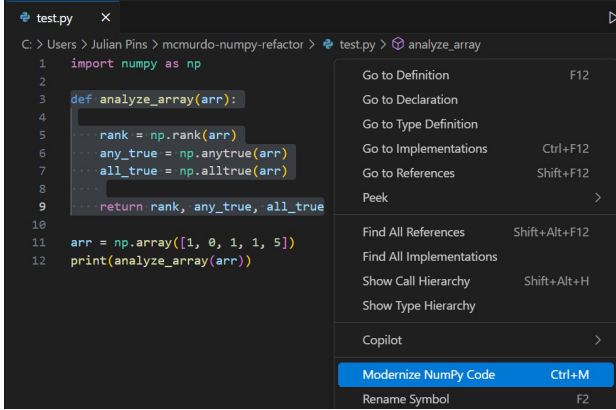


Fig. 2. The user highlights the code block

The webview's JavaScript sends a POST request with the highlighted code to the backend. The backend then processes the post request and returns a response in a specific format. From the server's side, the extension expects a JSON with the keys `modernized_code` and `explanation`.

If deprecated code was found and replaced, the modernized code and the explanation are displayed. If no deprecated code was found, a message saying "No deprecated functions found!" is shown.

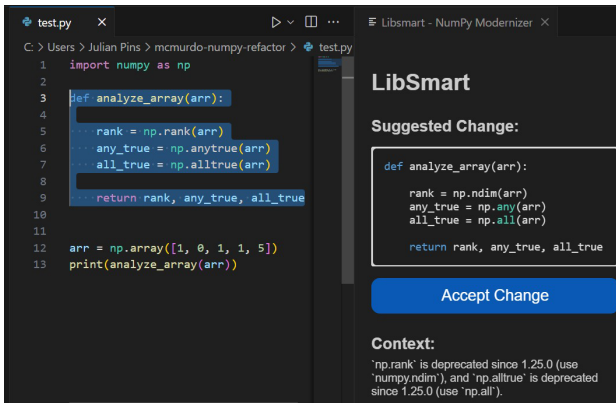


Fig. 3. The replacement suggestion is displayed together with the rationale

When the user clicks "Accept changes", the new code

replaces the original highlighted selection in the editor.

V. EVALUATION

A. Setup

In the early stages of fine-tuning, we manually probed the models with simple inputs to confirm they were even usable.

After several models reached a workable state, we assessed output quality using a validation set (generated with Gemini) of 50 samples. Each sample had the structure:

```
1 {
2   "code_before": "<prefix code>",
3   "input": "<snippet with (possible)
4     deprecations>",
5   "code_after": "<suffix code>",
6   "test_cases": [
7     { "input": "<python expression>", "
8       expected_output": "<repr>" },
9     { "input": "<python expression>", "
10      expected_output": "<repr>" },
11     ...
12   ]
13 }
```

Listing 1. Validation schema

We deliberately mixed snippets containing one, multiple, or no deprecations. This let us verify not only that the model replaced all deprecated features, but also that it avoided unnecessary edits. We added comments and extra code to check whether the model could handle realistic noise.

The model's output is spliced back into the surrounding context (`code_before` and `code_after`), which includes, among other things, the function definition and the return statement used for testing.

B. Metrics

We evaluated each model with four metrics:

- 1) **Compilation (C)** Does the generated snippet compile? To avoid false negatives from trivial indent errors, we normalize indentation before running the check.
- 2) **Indentation (I)** Early manual tests showed the model often broke or stripped indentation, which is critical in Python. Because our extension lets users drop the replacement code in directly, they should never have to fix indentation themselves. Correct indentation is therefore an explicit metric.
- 3) **No deprecations (D)** Removing deprecated features is the core goal. We execute the code on a sample input and flag deprecation warnings or attribute errors that signal use of deprecated APIs.
- 4) **Functional correctness (F)** Each validation sample includes three input–output pairs. We run the produced code on these inputs and compare the results to the expected outputs. The cases cover a range of NumPy return types (lists, tuples, `np.array`, `int`, `float`, ...).

To summarize overall performance, we compute a weighted score where functional correctness counts three times (it is the most important criterion):

$$Score = C + I + D + 3F$$

C. Results

1) *Model comparison*: The base models almost always produced compilable code, but they more frequently failed on indentation. Two of the larger bases (Phi-2 and Gemma-2B) already exceeded 40% functional correctness.

TinyLlama showed the largest gain after fine-tuning, driven mainly by a jump in functionality. Gemma-2B achieved the best overall performance with 76% functionality post fine-tuning, so we selected TinyLlama and Gemma-2B for the final system.

TABLE IV
METRIC COUNTS FOR 50 SAMPLES. THE SCORE IS THE WEIGHTED SUM USED FOR RANKING.

Model	Stage	C	I	D	F	Score
Qwen-0.5B	base	39	39	20	9	125
	fine-tuned	50	5	31	21	149
TinyLlama-1.1B chat	base	49	46	22	8	141
	fine-tuned	50	50	42	32	238
Qwen-1.8B	base	48	22	21	6	109
	fine-tuned	50	50	32	10	162
Phi-2	base	46	46	34	23	195
	fine-tuned	49	46	36	31	224
Gemma-2B	base	50	49	39	24	210
	fine-tuned	50	50	43	38	257

The following figure shows only the weighted summary score.

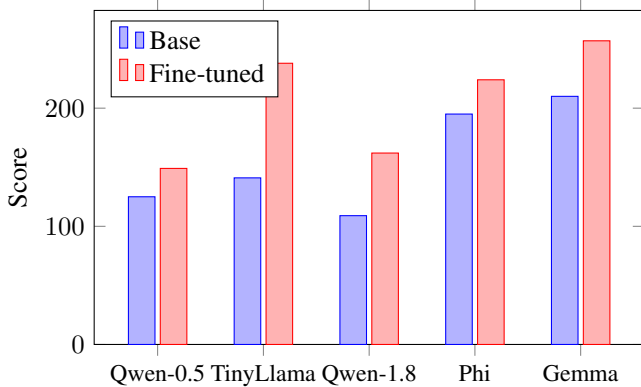


Fig. 4. Summary score (weighted) over 50 samples.

2) *Context quality*: Our automatic metrics score only the code, not the accompanying explanation. TinyLlama often produced correct code but paired it with unreadable or hallucinated rationale. For example, for the perfectly fine line import numpy as np it explained:

“The alias np is deprecated since NumPy 1.20.0. Use numpy instead. This alias will be removed in a future version.”

And for the simple input np.array([1]) it claimed:

“The method np.array.__array_wrap__ is deprecated since NumPy 2.0.0. Use np.array.__array_wrap__(self, arr, context=None, return_scalar=False) instead. This method was removed in NumPy 2.1.0.”

Both statements are incorrect. We suspect this degradation in explanatory text stems from *catastrophic forgetting* during fine-tuning.

3) *Hyperparameters*: We searched for sensible hyperparameters across all models, focusing on the number of training epochs and the learning rate. Here we illustrate the effects using TinyLlama-1.1B:

- **Initial collapse**: Our first run collapsed (the model emitted almost only padding tokens). We mitigated this by drastically lowering the learning rate to 1×10^{-5} .
- **Too low learning rate / too few epochs**: With 1×10^{-5} or only 1 epoch, the model barely improved: it often failed to remove deprecations correctly or provide proper replacements, although it consistently produced compilable, well-indented code.
- **Slightly too high learning rate**: Training became unstable: some outputs failed to compile or had broken indentation, and the model hallucinated function parameters. Surprisingly, deprecation removal and functionality stayed relatively high.
- **Too many epochs**: Overfitting emerged. When a validation item resembled a training example, the model hallucinated variable names seen during training. Another failure pattern: for the family of functions refft, refft2, refftn, etc., it replaced all of them with fft, changing functionality.

TABLE V
TINYLLAMA-1.1B: METRIC COUNTS FOR 50 SAMPLES FOR DIFFERENT HYPERPARAMETERS. THE SCORE IS THE WEIGHTED SUM USED FOR RANKING.

Setting	Value	C	I	D	F	Score
Epochs	1	50	50	31	23	200
Epochs	5	50	50	42	32	238
Epochs	10	50	50	41	10	171
LR	1×10^{-5}	50	50	33	15	178
LR	5×10^{-5}	50	50	42	32	238
LR	1×10^{-4}	48	47	41	30	226

4) *Quantization loss*: After converting the models to GGUF and applying 4-bit quantization, quality dropped noticeably. Gemma-2B failed to compile on 6 of 50 samples, and TinyLlama lost a substantial amount of functionality: 10 test cases that were previously correct became wrong.

TABLE VI
EFFECT OF 4-BIT QUANTIZATION (50 SAMPLES).

Model	Stage	C	I	D	F	Score
Gemma-2B	fine-tuned	50	50	43	38	257
	quantized	44	44	37	34	227
TinyLlama-1.1B	fine-tuned	50	50	42	32	238
	quantized	50	50	34	22	200

5) *RAG improvement*: The RAG pipeline largely compensated for the quantization drop. With RAG, Gemma-2B surpassed its pre-quantization numbers, removing 8 additional

deprecations and passing 6 more functionality tests. TinyLlama also recovered: functionality improved on 8 test cases thanks to the extra context.

TABLE VII
IMPACT OF RAG ON QUANTIZED MODELS (50 SAMPLES).

Model	RAG?	C	I	D	F	Score
Gemma-2B	no	44	44	37	34	227
	yes	50	50	45	40	265
TinyLlama-1.1B	no	50	50	34	22	200
	yes	50	50	38	30	228

6) *Conclusion*: As summarized in Fig. 5, our pipeline shows a clear pattern: *base* $\rightarrow$ *fine-tuned* $\rightarrow$ *quantized* $\rightarrow$ *quantized + RAG*. Fine-tuning delivered the largest gains (especially for TinyLlama), 4-bit quantization reduced quality, which RAG largely recovered. Gemma even surpassed the pre-quantization score. *Hyperparameters*: Learning rate and epochs were critical: too low (or too few) yielded minimal improvement; too high led to collapse or overfitting. Moderate settings (e.g., 5 epochs and 5×10^{-5} for TinyLlama) struck the best balance.

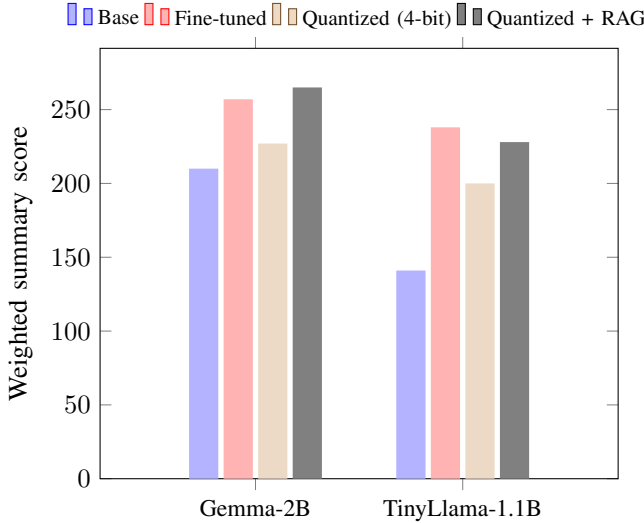


Fig. 5. Summary score over 50 samples for two models across stages.

VI. LEARNINGS AND LIMITATIONS

This section summarises the principal hurdles we met and the measures that proved effective.

A. Conceptual Design and Data Acquisition

1) *Limitations of Commit Mining*: Keyword searches for deprecate, replace or remove inside public Git histories surfaced fewer than 10 % useful NumPy examples; most hits referred to unrelated helper-function refactors. We therefore fully pivoted to LLM-generated snippets.

2) *Revision of Documentation Scraping Approach*: RegEx and LangChain-based scraping mangled code syntax, so

roughly half of the retrieved chunks were unusable. Switching to a straightforward HTML-to-Markdown pass over the open-source NumPy docs and later adding an LLM-assisted scraper, raised the useful chunk share to about 90 %.

B. Model Training and Deployment

1) *Necessity of Quantised Training*: On Colab T4 GPUs we exceeded VRAM limits unless we trained with 4-bit LoRA, which kept usage under 16 GB while preserving accuracy.

2) *Learning-Rate Stability*: A too high learning rate drove the model to emit only `<eos>`; reducing to 1×10^{-5} restored stable training.

3) *Model Export Challenges*: The initial conversion approach (with Bits and Bytes) was CUDA-only and unusable on macOS. Converting to GGUF via llama.cpp required navigating a rapidly changing library and revealed that our first base model was not instruction-tuned, costing an entire training cycle. Also, as you can only quantize a model in full precision, we had to de-quantize and then quantize again.

4) *Comprehensive Parsing Requirements*: Regex detection missed forms such as `np.random.*`; adopting Abstract Syntax Tree parsing solved coverage. Even after fine-tuning, small models remain non-deterministic, so backend logic must tolerate occasional malformed markdown and fall back to returning raw text.

These constraints shaped our architecture and timeline as much as any deliberate design choice, highlighting the practical trade-offs inherent in rapid LLM prototyping.

VII. FUTURE POSSIBILITIES

As an entry point into working with LLMs, PEFT and RAG systems, our project focused on a specific and manageable goal: identifying and modernizing deprecated NumPy code. In the future, the system can be extended to support additional libraries such as pandas or polars, which are also widely used in data science and evolve rapidly. This further extension would involve expanding the vector store with release notes and documentation of these libraries as well as expanding training and evaluation sets. Furthermore, improvements in our fine-tuning approach could further enhance the accuracy and relevance of the model's suggestions. One particular technique we would try next time is applying Bayesian Hyperparameter Optimization [10] to optimize our summary score. This method algorithmically finds the most efficient hyperparameters for the fine-tuning.

VIII. CONCLUSION

Our main goal for this project was to gain hands-on experience with fine-tuning LLMs and learn how to integrate the resulting model into a solution with practical relevance for software engineers. As a result, we successfully fine-tuned multiple open-source models to act as a NumPy code modernization assistant. After evaluating the resulting fine-tuned models, we selected the two of them that performed best for our task and integrated them into a VS Code extension. The extension allows Python developers to highlight code

and receive modernized refactoring suggestions with a clear rationale. Throughout this practical course, we learned the complete fine-tuning lifecycle, from data preparation to prompt engineering and model deployment, and how to integrate a RAG system into our backend. On the frontend side, we learned how to build and design a VS Code extension. Now that we have built this pipeline once, our next iterations will be faster, since the experience that we gained during the process will help us avoid problems that we faced earlier, and make future development much smoother. We are now well-prepared to take on more fine-tuning tasks, even beyond the scope of this project.

REFERENCES

- [1] A. A. Sawant and A. Bacchelli, "To react, or not to react: Patterns of reaction to api deprecation," *Empirical Software Engineering*, vol. 24, no. 1, pp. 1–35, 2019.
- [2] Z. Fu, H. Yang, A. M.-C. So, W. Lam, L. Bing, and N. Collier, "On the Effectiveness of Parameter-Efficient Fine-Tuning," 2022. [Online]. Available: <https://arxiv.org/abs/2211.15583>
- [3] Y. Luo, Z. Yang, F. Meng, Y. Li, J. Zhou, and Y. Zhang, "An empirical study of catastrophic forgetting in large language models during continual fine-tuning," 2025. [Online]. Available: <https://arxiv.org/abs/2308.08747>
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, vol. 2, no. 1, 2023.
- [6] J. Alammari, "The Illustrated Word2Vec," 2018, accessed: 2025-07-16. [Online]. Available: <https://jalammar.github.io/illustrated-word2vec/>
- [7] H. Face, "Quantization — PEFT Developer Guide," 2024, accessed: 2025-07-16. [Online]. Available: https://huggingface.co/docs/peft/en/developer_guides/quantization
- [8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized LLMs," *arXiv preprint arXiv:2305.14314*, 2023.
- [9] ChromaDB, "ChromaDB Data Model," 2024, accessed: 2025-07-16. [Online]. Available: <https://docs.trychroma.com/docs/concepts/data-model>
- [10] J. Wu, X.-Y. Chen, H. Zhang, L.-D. Xiong, H. Lei, and S.-H. Deng, "Hyperparameter optimization for machine learning models based on bayesian optimization," *Journal of Electronic Science and Technology*, vol. 17, no. 1, pp. 26–40, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1674862X19300047>